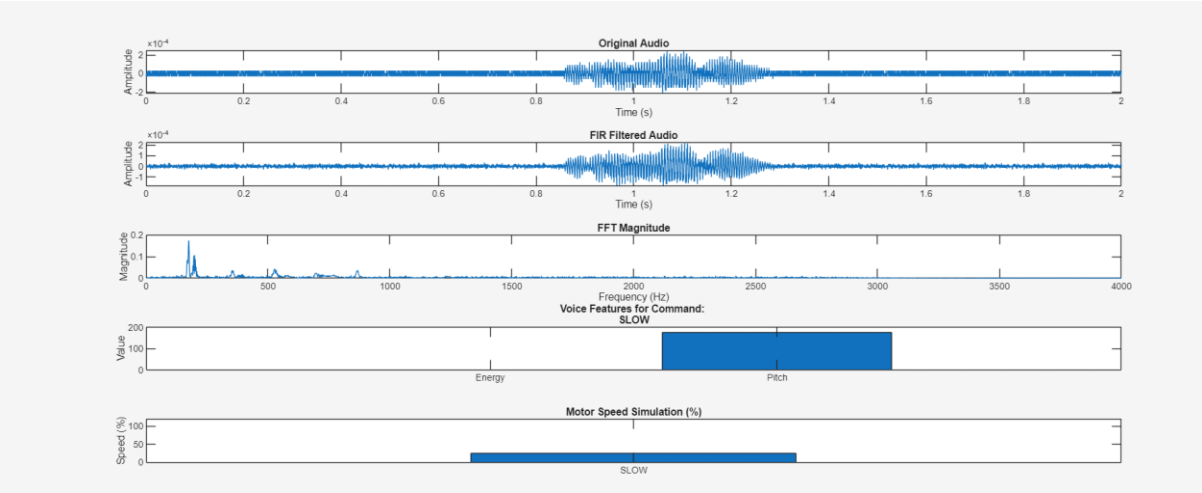
```
close all;
%% ----- Connect to ESP32 -----
esp32Port = "COM5"; % Set your ESP32 port
baud = 115200; % Baud rate
try
 esp32 = serialport(esp32Port, baud);
 configureTerminator(esp32,"LF"); % Send newline only
 flush(esp32); % Clear buffer
 disp(['✅ Connected to ESP32 on ', esp32Port]);
catch
 error("Cannot connect to ESP32 on COM5. Check connection.");
end
%% ----- Record Voice Command -----
fs = 16000; % Sampling frequency
recTime = 2; % Recording time in seconds
disp('🎙️ Speak a command: SLOW / FAST');
recObj = audiorecorder(fs,16,1); % Recorder object (16-bit, mono)
recordblocking(recObj, recTime); % Record for recTime seconds
y = getaudiodata(recObj); % Store recorded audio
%% ----- FIR Filtering -----
N = 50; % Filter order
fc = 3000; % Cutoff frequency (Hz)
b = fir1(N, fc/(fs/2)); % FIR low-pass filter coefficients
y_filt = filter(b,1,y); % Apply FIR filter
%% ----- Feature Extraction -----
energy = sum(y_filt.^2); % Signal energy
Y = abs(fft(y_filt)); % FFT magnitude
freqs = (0:length(Y)-1) * fs/length(Y); % Frequency vector
range = find(freqs >= 80 & freqs <= 300); % Pitch search range
[~,pitchIdx] = max(Y(range)); % Index of peak frequency
pitch = freqs(range(pitchIdx)); % Detected pitch frequency
%% ----- Classification -----
if energy < 0.05
 cmd = "SLOW";
```

```
else
 cmd = "FAST";
end
cmd = strtrim(cmd); % Remove spaces
cmd = upper(cmd); % Convert to uppercase (ESP32 format)
disp(['Detected Command: ', cmd]);
disp(['Energy = ', num2str(energy), ' | Pitch = ', num2str(pitch)]);
%% ----- Send Command -----
pause(0.1); % Small delay for buffer
writeline(esp32, cmd); % Send properly formatted command
disp("✅ Command sent to ESP32");
%% ----- Motor Speed Simulation -----
switch cmd
case "SLOW"
 motorSpeed = 25; % Example % duty cycle
case "FAST"
 motorSpeed = 100;
end
%% ----- Plotting -----
tVec = (0:length(y)-1)/fs; % Time vector
figure('Name','Voice Processing & Motor Control','NumberTitle','off');
% 1. Original Audio
subplot(5,1,1);
plot(tVec,y);
title('Original Audio');
xlabel('Time (s)');
ylabel('Amplitude');
% 2. FIR Filtered
subplot(5,1,2);
plot(tVec,y_filt);
title('FIR Filtered Audio');
xlabel('Time (s)');
ylabel('Amplitude');
% 3. FFT Magnitude
```

```
subplot(5,1,3);
plot(freqs,abs(Y));
title('FFT Magnitude');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
xlim([0 4000]);
% 4. Energy & Pitch Bar Chart
subplot(5,1,4);
bar([energy, pitch]);
set(gca, 'XTickLabel', {'Energy','Pitch'});
title(['Voice Features for Command: ', cmd]);
ylabel('Value');
% 5. Motor Speed Simulation
subplot(5,1,5);
bar(motorSpeed);
set(gca, 'XTickLabel', {cmd});
ylim([0 120]);
title('Motor Speed Simulation (%)');
ylabel('Speed (%)');
```

Output of the code:

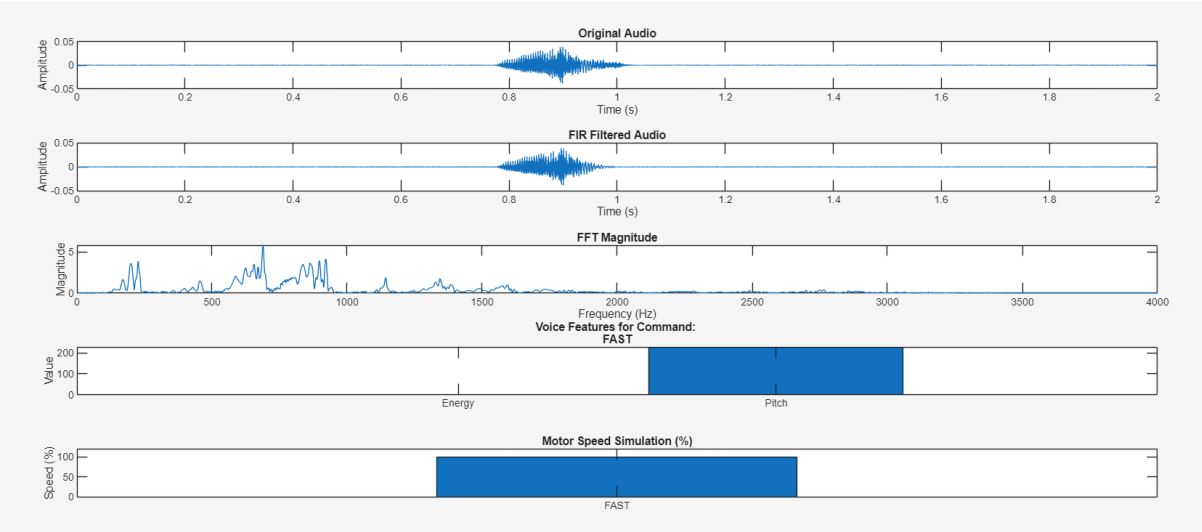
1]slow:




```
🎤 Speak a command: SLOW / FAST
   "Detected Command: "    "SLOW"

Energy = 3.6077e-05 | Pitch = 175
✅ Command sent to ESP32
```

2]fast:




```
🎤 Speak a command: SLOW / FAST
   "Detected Command: "    "FAST"

Energy = 0.29272 | Pitch = 226.5
✅ Command sent to ESP32
```

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ===== I2C LCD =====
LiquidCrystal_I2C lcd(0x27, 16, 4);

// ===== Motor Pins =====
const int ENA = 27; // PWM pin
const int IN1 = 25;
const int IN2 = 26;

// ===== PWM Setup =====
const int pwmChannel = 0;
const int freq = 5000;
const int resolution = 8;

// ===== Motor Timing =====
unsigned long motorStartTime = 0;
unsigned long motorDuration = 0;  // Duration for current command in ms
bool motorRunningTimed = false;

// ===== Current Motor Speed =====
int currentSpeed = -1;  // Track current PWM

// ===== Motor Durations (ms) =====
const unsigned long durationFAST = 5000;    // 5 seconds
const unsigned long durationMEDIUM = 4000;  // 4 seconds
const unsigned long durationSLOW = 3000;    // 3 seconds

// ===== Kickstart Duration =====
const int kickstartTime = 150;  // ms
```

```
const int kickstartPWM = 255;   // Full PWM for initial start

// ===== Motor Control Function =====
void setMotor(int speed, const String &label) {
  if (currentSpeed != speed) {
    if (speed > 0) {
      digitalWrite(IN1, HIGH);
      digitalWrite(IN2, LOW);

      // ===== Kickstart =====
      ledcWrite(pwmChannel, kickstartPWM);  // full power burst
      delay(kickstartTime);              // short burst to overcome static friction
      ledcWrite(pwmChannel, speed);        // settle to desired speed
    }
    else {
      digitalWrite(IN1, LOW);
      digitalWrite(IN2, LOW);
      ledcWrite(pwmChannel, 0);
    }

    // ===== LCD Update =====
    lcd.setCursor(0, 1);
    lcd.print("            ");   // Clear line
    lcd.setCursor(0, 1);
    lcd.print("Motor: " + label);

    // ===== Set motor duration based on label =====
    if (label == "FAST") motorDuration = durationFAST;
    else if (label == "MEDIUM") motorDuration = durationMEDIUM;
    else if (label == "SLOW") motorDuration = durationSLOW;
    else motorDuration = 0;

    motorStartTime = millis();
    motorRunningTimed = (speed > 0);
```

```
    currentSpeed = speed;
  }
}

void setup() {
  Serial.begin(115200);

  // LCD setup
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0,0);
  lcd.print("Voice Motor Ctrl");

  // Motor setup
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(ENA, pwmChannel);

  Serial.println("ESP32 Ready");
}

void loop() {
  // ----- Read Serial Command -----
  if (Serial.available()) {
    String command = Serial.readStringUntil('\n');
    command.trim();
    command.toUpperCase();

    Serial.print("DEBUG Received: [");
    Serial.print(command);
    Serial.println("]");

    // ----- Motor Control -----
```

```
    if (command.startsWith("FAST")) {
      setMotor(255, "FAST");
    }
    else if (command.startsWith("MEDIUM")) {
      setMotor(170, "MEDIUM");
    }
    else if (command.startsWith("SLOW")) {
      setMotor(100, "SLOW");
    }
    else if (command.startsWith("STOP")) {
      setMotor(0, "STOP");
    }
    else {
      lcd.setCursor(0, 1);
      lcd.print("Unknown CMD   ");
      currentSpeed = -1;  // reset for next valid command
    }
  }

  // ----- Auto-stop after set duration -----
  if (motorRunningTimed && millis() - motorStartTime >= motorDuration) {
    setMotor(0, "STOP");
  }
}
```